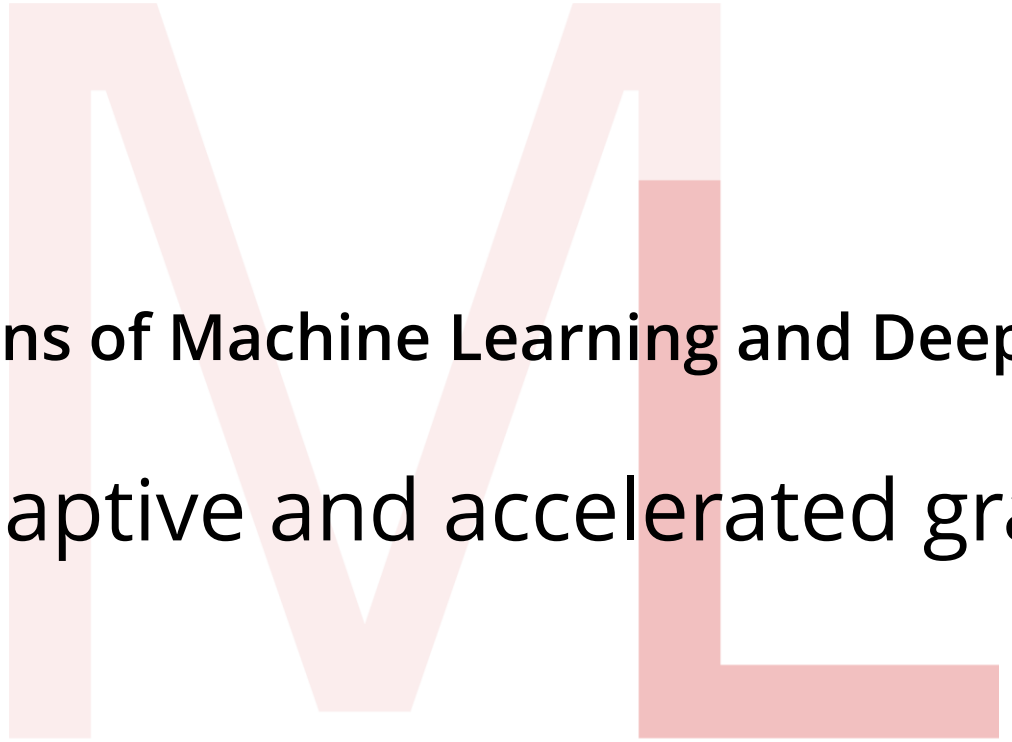
Foundations of Machine Learning and Deep Learning

Optimizers – Adaptive and accelerated gradient methods



upna
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Leaky averages

- In deep learning, the minibatch stochastic gradient descent can be calculated by

$$\boldsymbol{g}_{t,t-1} = \frac{1}{|B_t|} \sum_{i \in B_t} \boldsymbol{h}_{i,t-1}$$

- where $h_{i,t-1} = \partial_w f(x_i, w_{t-1})$ is the stochastic gradient descent for sample i using the weights updated at time $t-1$

Leaky averages

- Minibatch gradient descent reduces variance with respect to stochastic gradient descent
- The variance can be reduced even more by replacing the gradient computation by a "leaky average"

$$\boldsymbol{v}_t = \beta \boldsymbol{v}_{t-1} + \boldsymbol{g}_{t,t-1}$$

- For some $\beta \in (0,1)$. This replaces the instantaneous gradient by one averaged over multiple past gradients
- v is called velocity
- Recursively:

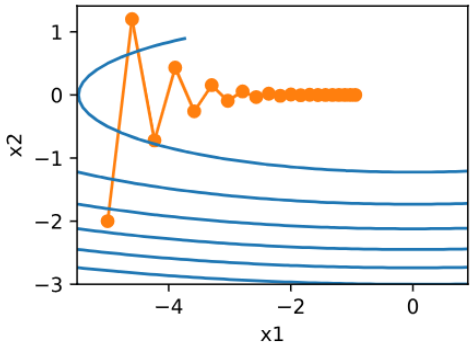
$$\mathbf{v}_t = \beta^2 \mathbf{v}_{t-2} + \beta \mathbf{g}_{t-1,t-2} + \mathbf{g}_{t,t-1} = \cdots, = \sum_{\tau=0}^{t-1} \beta^\tau \, \mathbf{g}_{t-\tau,t-\tau-1}.$$

The momentum method

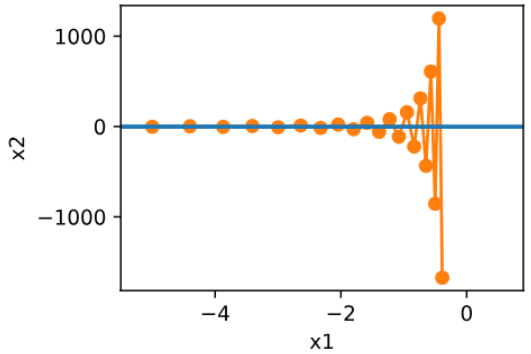
- Update the parameters using the velocity instead of using the gradient directly

$$\boldsymbol{v}_t \leftarrow \beta \boldsymbol{v}_{t-1} + \boldsymbol{g}_{t,t-1}$$
$$\boldsymbol{x}_t \leftarrow \boldsymbol{x}_{t-1} + \eta_t \boldsymbol{v}_t$$

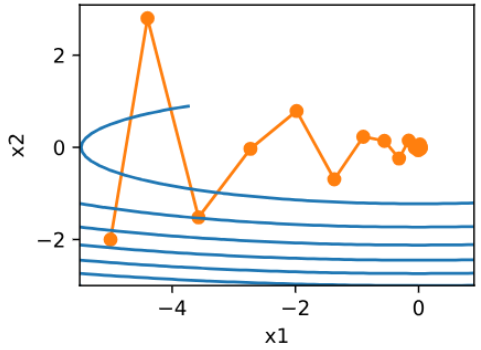
Example: $f(x) = 0{,}1x_1^2 + 2x_2^2$



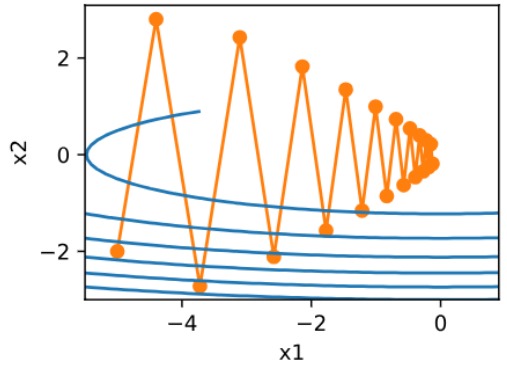
Gradient descent
20 epochs
$\alpha = 0{,}4$



Gradient descent
20 epochs
$\alpha = 0{,}6$



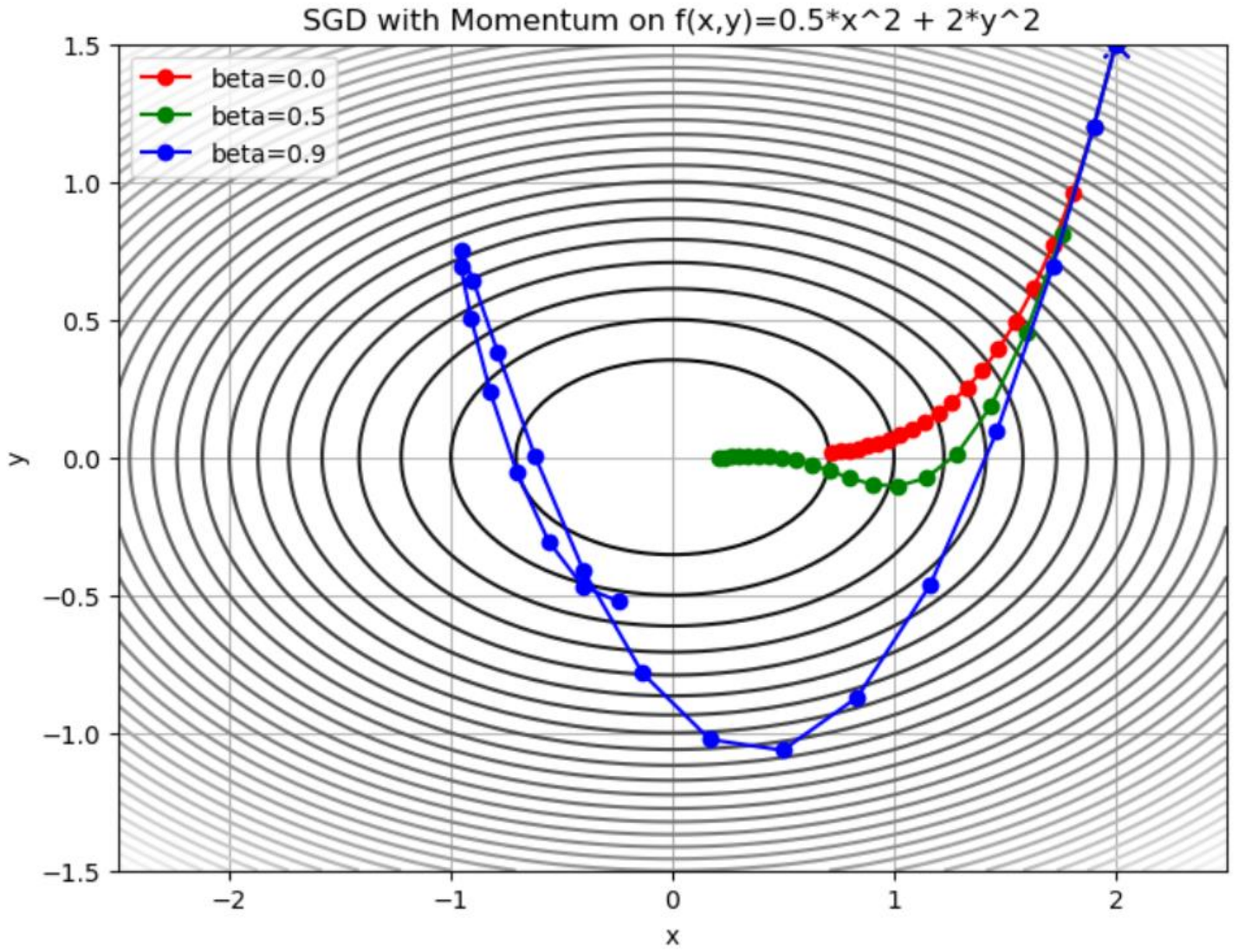
Momentum
20 epochs
$\alpha = 0{,}6$
$\beta = 0{,}5$



Momentum
20 epochs
$\alpha = 0{,}6$
$\beta = 0{,}25$

The momentum method

- Example:
  - $f(x, y) = 0.5x^2 + 2y^2$
  - Initial point: $(2, 1.5)$
  - $\alpha = 0.05$
  - steps=20



AdaGrad

- Newton's method uses the Hessian matrix to preconditionate the gradient
  - Computationally very expensive
  - Use only the diagonal of the matrix
- Adagrad also uses the idea of preconditioning with a diagonal matrix
  - Each value of the diagonal is related to one dimension
  - Each value adjust the learning rate in its dimension

AdaGrad

- At a given time
  - if one dimension was updated frequently with big values (is already close to the objective) → need smaller learning rate
  - If one dimension was rarely updated or updated with very small values (is still far from the objective) → need bigger learning rate

- Idea: aggregate the past gradients in each variable
  - To stress the effect, the gradients are squared

AdaGrad

- Initialization of accumulated gradient

$$\boldsymbol{s}_t = \mathbf{0}$$

- At each step t
  - Calculate the gradient

$$\boldsymbol{g}_t = \partial_{\boldsymbol{w}} l(y_t, f(\boldsymbol{x}_t, \boldsymbol{w}))$$

  - Update the accumulated gradient

$$\boldsymbol{s}_t = \boldsymbol{s}_{t-1} + \boldsymbol{g}_t^2$$

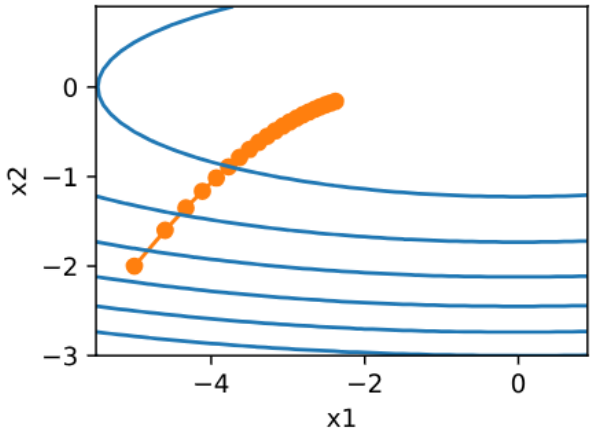
  - Update the parameters

$$\boldsymbol{w}_t = \boldsymbol{w}_{t-1} - \frac{\eta}{\sqrt{\boldsymbol{s}_t + \epsilon}} \cdot \boldsymbol{g}_t \quad (\epsilon \text{ to avoid division by } 0)$$

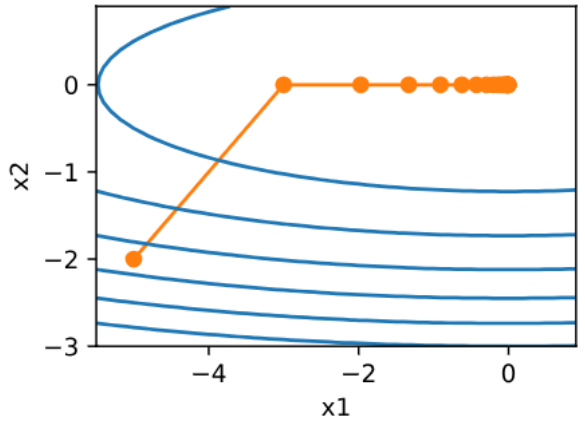
- The update is in the direction of the gradient, scaled by the accumulated gradient

AdaGrad

- Example:
  - $f(x, y) = 0{,}1x^2 + 2y^2$
  - Initial point: (-5, -2)



$\eta = 0{,}4$



$\eta = 2$

RMSProp

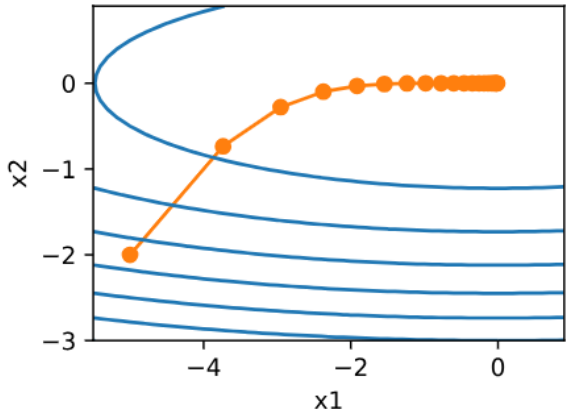
- In Adagrad, $\boldsymbol{s}_t$ (accumulated grad) is monotone increasing, so steps are smaller at every step (it can stop the learning too fast)
- Idea: use a leaky average

$$\boldsymbol{s}_t \leftarrow \gamma \boldsymbol{s}_{t-1} + (1-\gamma)\boldsymbol{g}_t^2$$

- The update of the parameters is the same as Adagrad

$$\boldsymbol{w}_t = \boldsymbol{w}_{t-1} - \frac{\eta}{\sqrt{\boldsymbol{s}_t + \epsilon}} \odot \boldsymbol{g}_t$$

- Example:
  - $f(x, y) = 0{,}1x^2 + 2y^2$
  - Initial point: (-5, -2)
  - $\eta = 0.4; \gamma = 0.9$



AdaDelta

- Uses two state variables to store:
  - a leaky average of square gradients
  - A leaky average of updates of the parameters
- Replaces the fixed η by the leaky averages of updates
  - To stress the effect, the updates are squared

$$\boldsymbol{s}_t = \rho \boldsymbol{s}_{t-1} + (1-\rho)\boldsymbol{g}_t^2$$

$$\boldsymbol{x}_t = \boldsymbol{x}_{t-1} - \boldsymbol{g}'_t$$

$$\boldsymbol{g}'_t = \frac{\sqrt{\Delta \boldsymbol{x}_{t-1} + \epsilon}}{\sqrt{\boldsymbol{s}_t + \epsilon}} \odot \boldsymbol{g}_t$$

$$\Delta \boldsymbol{x}_t = \rho \Delta \boldsymbol{x}_{t-1} + (1-\rho)\boldsymbol{g}'^2_t$$

Adam (Adaptive Moment Estimation)

- Combines some of the techniques proposed in previous optimizers
  - Momentum
  - Preconditioning of RMSProp
  - Normalization of leaky averages using $\sum_{i=0}^{t-1} \beta^i = \frac{1-\beta^t}{1-\beta}$

- Momentum

$$\boldsymbol{v}_t \leftarrow \beta_1 \boldsymbol{v}_{t-1} + (1-\beta_1)\boldsymbol{g}_t \quad \Rightarrow \quad \hat{\boldsymbol{v}} = \frac{\beta_1 \boldsymbol{v}_{t-1} + (1-\beta_1)\boldsymbol{g}_t}{1-\beta_1^t}$$

- Accumulated grad

$$\boldsymbol{s}_t \leftarrow \beta_2 \boldsymbol{s}_{t-1} + (1-\beta_2)\boldsymbol{g}_t^2 \quad \Rightarrow \quad \hat{\boldsymbol{s}} = \frac{\beta_2 \boldsymbol{s}_{t-1} + (1-\beta_2)\boldsymbol{g}_t^2}{1-\beta_2^t}$$

Adam (Adaptive Moment Estimation)

- Update equations
  - Beware that the ϵ in the denominator of this method is out of the square root

$$\boldsymbol{g}'_t = \frac{\eta \widehat{\boldsymbol{v}_t}}{\sqrt{\widehat{\boldsymbol{s}_t}} + \epsilon}$$
$$\boldsymbol{x}_t \leftarrow \boldsymbol{x}_{t-1} - \boldsymbol{g}'_t$$

- Typical values
  - $\beta_1 = 0.9$
  - $\beta_2 = 0.999$
  - $\epsilon = 10^{-6}$

Optimizers review

- **Gradient Descent (Batch, Mini-batch, Stochastic)**
  - Idea: update parameters in the direction of the gradient
  - Variants:
    - Batch: uses the full dataset → stable but expensive
    - Stochastic: uses a single sample → noisy but cheap
    - Mini-batch: practical compromise, most commonly used
  - Limitation: a single global Learning rate may be inefficient in ill-conditional landscapes
- **Momentum**
  - Idea: introduces a “velocity” term that accumulates past gradients
  - Benefit: smooths oscillations, accelerates learning in consistent directions
  - Intuition: like a ball Rolling downhill with inertia
- **Adagrad**
  - Idea: scales each parameter individually based on the historical sum of squared gradients
  - Benefit: great for sparce data (rare parameters get larger updates)
  - Limitation: Learning rate per coordinate keeps shrinking, may “stall” training

Optimizers review

- **RMSProp**
  - Idea: similar to Adagrad but uses an exponential moving (leaking) average instead of an ever-growing sum
  - Benefit: Avoids the “stalling” problem
  - Limitation: still requires a global Learning rate
- **AdaDelta**
  - Idea: Extends RMSProp by also keeping a leaking average of parameter updates
  - Benefit: eliminates the need for a fixed global Learning rate
  - Intuition: Adapts the step size based on the scale of recent updates
- **Adam**
  - Idea: combines momentum + RMSProp with bias correction
  - Benefit: very popular, robust and effective in most cases
  - Limitation: may converge to suboptimal solutions; sometimes SGD+momentum generalizes better

Learning rate schedulers

- Most of the optimizers need a learning rate
- A constant learning rate may not be adequate for all the steps
- Scheduler: a function that calculates the learning rate based on the number of updates already done
- Intuition:
  - Fix a starting learning rate η_0
  - Define a policy to adjust η through time
  - At each batch/epoch update $\eta \leftarrow f(t)$

Policies

- Polynomial decay

$$\eta_t = \eta_0 \left(1 - \frac{t}{T}\right)^p$$

  - Adjust the velocity of the decay with power p

- Factor scheduler

$$\eta_{t+1} = \eta_t \alpha, \qquad \alpha \in (0,1)$$

  - It can be lower bounded: $\eta_{t+1} = \max(\eta_t \alpha, \eta_{min})$

- Multifactor scheduler (piecewise constant and decrease by a factor)

$$s = set\ of\ change\ points \qquad \eta_{t+1} = \eta_t \alpha, whenever\ t \in s$$

  - Every s epochs, make a crisp decay multiplying by a factor

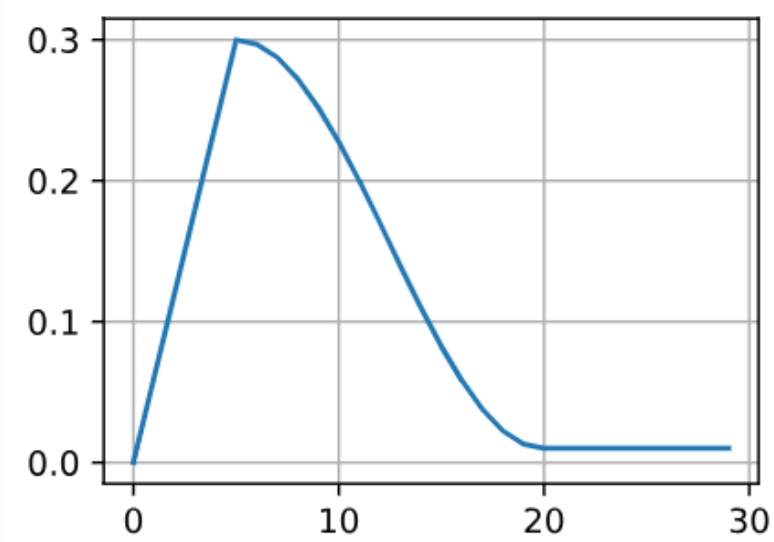
- Cosine scheduler

$$\eta_t = \eta_T + \frac{\eta_0 - \eta_T}{2}(1 + \cos\left(\frac{\pi t}{T}\right))$$

  - η_T target rate at T. For $t > T$, pin the value to η_T

Warmup

- Sometimes, initializing the parameters is not enough to guarantee a Good solution
- Warmup period during which the Learning rate increases to its initial máximum and to cool down the rate until the end of the optimization process
  - Can be applied to any scheduler



Learning rate schedulers

- Some schedulers work better if used per epoch
- Some schedulers have to be used per batch

- Some schedulers also adjust momentum
- Some optimizers adapt internally LR

- Study in detail the specifications of the optimizer and learning rate scheduler selected!!

```python
import torch
import torch.optim as optim
import torch.nn as nn

model = nn.Linear(10, 2)
optimizer = optim.SGD(model.parameters(), lr=0.1)
scheduler = optim.lr_scheduler.MultiStepLR(optimizer, milestones=[30, 80], gamma=0.1)

for epoch in range(100):
    # Training code here
    scheduler.step()

```